

CS 450: Assignment 04

Programming Assignments (95%)

- This assignment is in C++ and GLSL.
- **Main application:**
 - o Copy *src/app/Assign03.cpp* and name it *src/app/Assign04.cpp*
 - o Change the **application name** from "Assign03" to "Assign04"
 - o Change the **window title** to "Assign04: " + your SITNET ID
 - E.g., "Assign04: realemj"
 - o Follow the exercise instructions from: **11 "Matrix Transformations"**
 - Add any code for **Exercises11.cpp** into **Assign04.cpp**
 - o You will also draw from code in **09 "Scene Graph and Assimp"**
 - o **Add the following includes:**
 - `#include <assimp/Importer.hpp>`
 - `#include <assimp/scene.h>`
 - `#include <assimp/postprocess.h>`
 - o **Add a `glm::vec4` color to your `ForgeVertex` struct**
 - o Create a new function:
`void extractMeshData(aiMesh *mesh, HostMesh<ForgeVertex> &m)`
 - The overall purpose of this function to grab the vertex positions and shape indices from the given `aiMesh` and store this information in the provided `HostMesh` struct.
 - Clear out the `HostMesh`'s vertices and indices.
 - Loop through all vertices in the `aiMesh` (`mesh->mNumVertices`):
 - Create a `ForgeVertex`.
 - Grab the vertex position information from `mesh->mVertices[i]` and store it in the `ForgeVertex`'s position.
 - o Note: `mVertices` is an array of `aiVector3D` structs, so you will have to somehow convert to `glm::vec3` structs
 - o Set the color of the `ForgeVertex` to **any color other than (0,0,0,1) or the background.**
 - You may use different colors per vertex using any reasonable scheme.
 - However, alpha values should always be 1.0f.
 - Add the `ForgeVertex` to the `HostMesh`'s vertices list.
 - Loop through all faces in the `aiMesh` (`mesh->mNumFaces`):
 - Grab the `aiFace` *face* from `mesh->mFaces[i]`.
 - Loop through the number of indices for this face (`face.mNumIndices`):

- Add each index for the face (face.mIndices[k]) to the HostMesh's list of indices.
- **Change recordCommands():**
 - Have it take a **vector<VulkanMesh> &allMeshes** INSTEAD of a single VulkanMesh &mesh.
 - Instead of a single call to recordDrawVulkanMesh(), loop through allMeshes and draw **every** mesh.
- **In the main function:**
 - **The model to load will be provided on the command line:**
 - Use "sampleModels/sphere.obj" as your default model path.
 - If argc >= 2, grab argv[1] as the model path to load and convert to a string.
 - **NOTE: Do NOT use cin!! You MUST use the command line arguments!**
 - **Load the model (given by the model path) using Assimp to get an aiScene**
 - Use the following options (OR'ed together):
 - aiProcess_Triangulate
 - aiProcess_FlipUVs
 - aiProcess_GenNormals
 - aiProcess_JoinIdenticalVertices
 - **Check to make sure the model loaded correctly**
 - Print error and exit if ANY of these are true:
 - The scene object is null
 - scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE
 - scene->mRootNode is null
 - **Make sure to add color to the pipeline's list of VertexInputAttributeDescription's**
 - ```
pipelineCreateInfo.attribDesc.push_back(
vk::VertexInputAttributeDescription(
1, // location
0, // binding
vk::Format::eR32G32B32A32Sfloat, // format
offsetof(ForgeVertex, color) // offset
));
```
  - **Comment out the previous HostMesh creation code.**
  - **Create a C++ vector of VulkanMesh's**
    - Add item: myVector.push\_back(thing);
    - Number of items: myVector.size()
    - Get item: myVector.at(i) OR myVector[i]
  - **Before your drawing loop, in the staging area:**
    - For each **mesh** in the Assimp scene (mMeshes with mNumMeshes):

- Create a **HostMesh<ForgeVertex>** object inside the loop
  - Call **extractMeshData** to get a **HostMesh** from each scene->mMeshes[i]
  - Call **createVulkanMesh()** to get a **VulkanMesh**
  - Call **copyToVulkanMesh()**
  - Add the **VulkanMesh** to your **vector of VulkanMesh's**
- **In your drawing loop:**
  - Pass in your list of VulkanMesh objects to recordCommands()
- **After your drawing loop:**
  - Comment out previous cleanupVulkanMesh call
  - Loop through all of your VulkanMesh objects and call cleanupVulkanMesh()
  - Clear out your list of VulkanMesh objects
- **Shader code:**
  - Copy folder **vulkanshaders/Assign03** as **vulkanshaders/Assign04**
  - Follow the exercise instructions from: **11 "Matrix Transformations"** for the shader code
  - In addition, **ensure that color information is correctly input by the vertex shader, transferred to the fragment shader, and used as the output color in the fragment shader.**
    - **shader.vert:**
      - Add input color for each vertex:  
**layout(location = 1) in vec4 color;**
      - Add an output color:  
**layout(location = 0) out vec4 interColor;**
      - In main(), forward color information to fragment shader:  
**interColor = color;**
    - **shader.frag:**
      - Add input color for each fragment:  
**layout(location = 0) in vec4 interColor;**
      - Set output color to fragment color:  
**out\_color = interColor;**
- **CMake setup:**
  - Modify **CMakeLists.txt** by adding the following line to the end of the file:
    - **CREATE\_VULKAN\_EXECUTABLE(Assign04)**

## Screenshot (5%)

Sample models have been provided for this assignment. To pass in command line arguments:

- Go to .vscode/launch.json
- Change "args" entry to [ "./sampleModels/teapot.obj" ],

To test NO command line arguments:

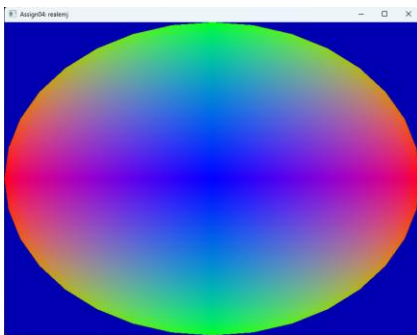
- Change "args" entry to [ ]

**Remember: your program should load `./sampleModels/sphere.obj` by default if no command line arguments are provided.**

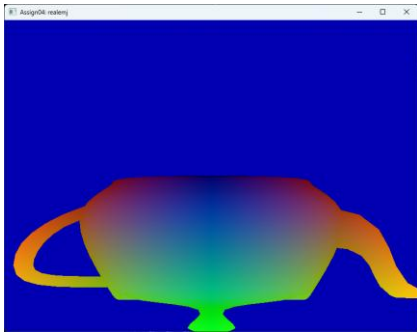
For this part of the assignment, **upload TWO screenshots** of the application window (note that window title should be "Assign04: yoursitnetid"):

- One displaying `sphere.obj` → `Assign04_sphere.png`
- One displaying `teapot.obj` → `Assign04_teapot.png`

*sphere.obj (color need not match this image):*



*teapot.obj (color need not match this image):*



Copy these images into the **screenshots** folder